ITA06 MACHINE LEARNING

ASSIGNMENT 1

Name : Tamil Elakiya S

Reg No : 192524443

CO1: Learning problems and applications, Perspectives and issues in machine learning, Types of learning, Concept learning task, Version spaces, Candidate elimination algorithm, Decision tree learning algorithm, Heuristic search in decision trees, Overfitting and underfitting

---

Assignment Title: Design, Analyse and Validate an Intelligent Early-Warning Decision-Support System for Academic Risk Identification Using Concept Learning and Decision Tree Learning

(**Total Marks:** 50)

Duration: 1 Week

Scenario – University Academic Early-Warning System

A university wishes to identify, early in the semester, students who are **at risk of failing** so that the academic office can offer timely counselling, remedial classes or mentoring. Advisors currently flag students manually, which is slow, inconsistent between advisors and not scalable across thousands of students.

The university has a small, labelled history of past students. For each student, the following **categorical attributes** are recorded after week 6 of the semester :

Attribute	Values	Operational meaning
Attendance	Low, Medium, High	Low < 65 %; Medium 65–80 %; High > 80 %
Assignments	Irregular, Regular	Regular = at least 80 % submitted on time
StudyHours	Low, High	Self-reported hours per week: Low < 8 h
PriorCGPA	Weak, Strong	Weak < 6.0; Strong ≥ 6.0
LMS	Passive, Active	Activity on the learning-management system
Mentoring	No, Yes	Enrolled in a mentoring programme

TASK 1 – Identify and Formulate the ML Problem

1. Formal learning problem (T, P, E)

Element	Statement
Task (T)	Given the six categorical attributes recorded after week 6, classify a student as AtRisk = Yes or No, with a human-readable reason for the decision.
Performance (P)	Primary: **recall** of the at-risk class on unseen students (council target ≥ 90 %). Secondary: precision, F1 and accuracy, measured on held-out data and by cross-validation.
Experience (E)	Labelled history of past students: Dataset A (5), Dataset B (14), Test set T (6), later about 2,000 records. Each label is the academic council's end-of-semester decision.

2. Attribute table

Attribute	Type	Values	Role	Why it is plausibly relevant
Attendance	Ordinal categorical	Low, Medium, High	Input	Missed classes mean missed content; strongest behavioural early signal.
Assignments	Binary	Irregular, Regular	Input	Late or missing work shows disengagement and loses internal marks.
StudyHours	Binary	Low, High	Input	Time on task drives learning; (Dataset A only).
PriorCGPA	Binary	Weak, Strong	Input	Past performance is a proxy for foundation and study skills.
LMS	Binary	Passive, Active	Input	Online activity reflects ongoing engagement with material.
Mentoring	Binary	No, Yes	Input	Support received may reduce risk (Dataset A only).
AtRisk	Binary	Yes, No	Target	Council decision at end of semester.

Poor or risky predictor: StudyHours. It is self-reported, so it is subject to over-reporting and recall bias, the threshold of 8 h is arbitrary and the same hours can be of very different quality. It is also correlated with Attendance and LMS, so it adds little independent information. (Mentoring is also risky because it is an intervention applied after risk is noticed, which creates leakage and a misleading link to the outcome.)

3. Target concept and instance space

The target concept is a Boolean function $c : X \rightarrow \{0, 1\}$, where X = Attendance × Assignments × StudyHours × PriorCGPA × LMS × Mentoring and $c(x) = 1$ if and only if AtRisk(x) = Yes.

Dataset	Calculation	|X|	Distinct training examples	Coverage
A (6 attributes)	3 × 2 × 2 × 2 × 2 × 2	96	5	5 / 96 = 5.21 %
B (4 attributes)	3 × 2 × 2 × 2	24	14	14 / 24 = 58.33 %
B + T (4 attributes)	same space as B	24	20	20 / 24 = 83.33 %

Dataset A leaves 94.8 % of its space unseen, so any learner must generalise by assumption (bias). Dataset B sees more than half of its space.

4. Learning paradigm

Supervised learning (classification). Every record has a known label (AtRisk assigned by the council), the output is a discrete class, and the goal is to predict that label for new students. Features are categorical and the examples are independent. There is no sequential interaction or reward, which rules out reinforcement learning, and the labels make unsupervised learning unnecessary for the core decision.

5. Machine-learning issues

Issue	Where it appears in these datasets / application	Mitigation
Small data	Dataset A: 5 examples for 96 instances; Dataset B: 14 examples. Models have high variance (a fully grown tree reached 100 % train but only 50 % test).	Cross-validation, depth limits / pruning, collect more records.
Label noise	Several advisors assign labels; audit estimates 5–10 % inconsistent (Task 11). E6 contradicts E1 exactly.	Noise-tolerant stopping, pruning, adjudicate conflicting records.
Sampling bias / coverage	B covers 14 of 24 combinations; T contains combinations (and labels) unlike B; new values such as "Very Low" appear.	Stratified sampling, back-off policy for unseen values, regular refresh.
Generalisation	Full tree has 8 leaves for 14 students; leaves hold 1–3 students.	Control tree size; judge by held-out performance, not training accuracy.
Fairness and privacy	PriorCGPA and attendance can disadvantage students from weaker schools or with health or caring duties; records are personal.	Audit error rates by group, anonymise data, restrict access, human review of flags.
Measurement validity	StudyHours is self-reported; Mentoring is assigned after risk is noticed (leakage).	Drop or replace unreliable attributes; use only data available at week 6.

6. Which error is costlier?

A **false negative** (missing an at-risk student) is costlier: the student receives no counselling or remedial help and may fail a semester, which is hard to undo. A **false positive** costs only a counselling slot and a short conversation, which is cheap and reversible. I therefore emphasise **recall (sensitivity) of the at-risk class**, while monitoring precision and F1 so that the advisors are not overloaded with unnecessary flags.

TASK 2 – Analyse the Learning Paradigm Marks: 3

1. Comparison matrix

Criterion	Supervised	Unsupervised	Reinforcement
Labels available?	Yes – council label AtRisk for every past student	Labels ignored / not needed	No labels; only a reward after actions
Form of output	Class (Yes/No) or class probability	Clusters or profiles (no risk meaning)	A policy: which action in which state
Feedback signal	Error against the true label	None (internal cohesion only)	Delayed reward (pass/fail minus cost)
Data needed	Hundreds to thousands of labelled students	Many unlabelled students	Many interaction episodes with students
Success evaluation	Recall, precision, F1, confusion matrix	Cluster quality; no ground truth for risk	Cumulative reward; needs a simulator or trial
Interpretability for advisors	High for trees and rules	Medium (profile descriptions)	Low (value functions and policy)
Main risk	Label noise and bias copied from advisors	Clusters may not match risk	Unethical experimentation, sparse delayed reward

2. Unsupervised learning in this scenario

K-modes (or hierarchical) clustering of the six categorical attributes could group students into behavioural profiles, for example "low attendance, passive on LMS, irregular" or "regular, active, strong CGPA". Advisors could use these to design programmes. It **cannot** decide who is at risk: a cluster has no label, so nothing tells us which cluster the council would fail, no recall can be computed, and a student cannot be given a reason in terms of the council's outcome. Cluster boundaries also have no link to a pass/fail threshold.

3. Reinforcement learning (follow-up problem)

State: week number, attendance level, assignment status, LMS activity, current risk score, interventions already given. **Action:** {no action, reminder e-mail, mentoring, remedial class, counselling meeting}. **Reward:** +1 if the student passes, −1 if the student fails, minus a small cost for each intervention used. RL is unsuitable for the initial classification because: (i) the decision is a single prediction, not a sequence of actions that change the state; (ii) labelled outcomes already exist, so trial-and-error is wasteful; (iii) exploring random actions on real students is ethically unacceptable; (iv) the reward arrives only at semester end and is very sparse.

4. Selected paradigm, counter-argument and rebuttal

Selection: supervised learning, because labels exist, the output is a class, the system must be evaluated by recall and must explain its decisions. **Strongest argument against:** the labels come from several advisors and contain 5–10 % noise, so a supervised model learns human inconsistency. **Rebuttal:** the label is the council's end-of-semester outcome, not an advisor's guess, and noise of this size can be handled by pruning and noise-tolerant stopping (my Dataset C experiment: at 10 % noise the controlled tree still scores 100 % on clean validation data). Unsupervised methods cannot be evaluated for recall at all.

5. Hybrid proposal

Use **unsupervised clustering to create behavioural profile features and drift detection**, then feed profile plus attributes into the **supervised tree**. Benefit: advisors get a readable profile ("online, low-engagement") beside the rule, and a new cluster appearing (such as the online-mode students with Attendance = Very Low) alerts the university early that the model is facing data it has not learned. A second hybrid is supervised risk scores driving an RL scheduler for interventions.

TASK 3 – Formulate the Concept Learning Task **Marks: 3**

1. Target concept and labelled examples

$c : X \rightarrow \{0, 1\}$, $c(x) = 1$ if AtRisk(x) = Yes. Order of attributes: ⟨Attendance, Assignments, StudyHours, PriorCGPA, LMS, Mentoring⟩.

#	Instance	AtRisk	c(x)	Type
E1	⟨Low, Irregular, Low, Weak, Passive, No⟩	Yes	1	Positive
E2	⟨Low, Irregular, Low, Strong, Passive, No⟩	Yes	1	Positive
E3	⟨High, Regular, High, Strong, Active, Yes⟩	No	0	Negative
E4	⟨Low, Irregular, High, Weak, Passive, Yes⟩	Yes	1	Positive
E5	⟨Medium, Irregular, High, Strong, Active, No⟩	No	0	Negative

2. Hypothesis representation

A hypothesis is a conjunction of six constraints, each a specific value, "?" (any value) or "∅" (no value).

- Most general hypothesis: **⟨?, ?, ?, ?, ?, ?⟩** (classifies every instance as positive).
- Most specific hypothesis: **⟨∅, ∅, ∅, ∅, ∅, ∅⟩** (classifies every instance as negative).
- E1 as a hypothesis: ⟨Low, Irregular, Low, Weak, Passive, No⟩.
- E3 as a hypothesis: ⟨High, Regular, High, Strong, Active, Yes⟩.

3. Size of the hypothesis space (Dataset A)

Quantity	Calculation	Value
(i) Syntactically distinct hypotheses	Each attribute has (values + "?" + "∅"): $(3+2)(2+2)(2+2)(2+2)(2+2)(2+2) = 5 \times 4^5$	5,120
(ii) Semantically distinct hypotheses	All hypotheses containing ∅ classify nothing as positive, so they are one: $1 + (3+1)(2+1)^5 = 1 + 4 \times 243$	973
(iii) Instance space	$3 \times 2 \times 2 \times 2 \times 2 \times 2$	96

4. Three hypotheses with an ordering

Hypothesis	Relation and justification
h1 = ⟨Low, ?, ?, ?, Passive, ?⟩	h1 ≥g h2: every instance covered by h2 has Attendance = Low and LMS = Passive, so h1 covers it too. The reverse is false (h1 covers Low-attendance Regular students; h2 does not).
h2 = ⟨Low, Irregular, ?, ?, Passive, ?⟩	More specific than h1 because it adds Assignments = Irregular.
h3 = ⟨?, ?, High, ?, ?, Yes⟩	Incomparable with h1 and h2: ⟨High, Regular, High, Strong, Active, Yes⟩ is covered by h3 but not h1; ⟨Low, Irregular, Low, Weak, Passive, No⟩ is covered by h1 and h2 but not h3. Neither covers the other's instances.

5. Consistency with Dataset A

Hypothesis	Covers	Result	Reason
h_a = ⟨Low, ?, ?, ?, Passive, ?⟩	E1, E2, E4	**Consistent**	Covers all three positives and neither negative (E3, E5 are Active).
h_b = ⟨?, Irregular, ?, ?, ?, ?⟩	E1, E2, E4, E5	**Inconsistent**	Covers negative **E5** (Medium, Irregular, …) – a false positive.
h_c = ⟨Low, Irregular, Low, ?, ?, ?⟩	E1, E2	**Inconsistent**	Positive **E4** (StudyHours = High) is not covered – a false negative.

6. Satisfies versus consistent

A hypothesis **satisfies an instance** if its constraints match that single instance (h(x) = 1), for example h_b satisfies E5. A hypothesis is **consistent with the training data** only if h(x) = c(x) for every training example. So h_b satisfies E1, E2, E4 and E5 yet is inconsistent, because satisfying the negative E5 is an error; h_c satisfies E1 and E2 but fails to satisfy the positive E4. Satisfaction is a property of one instance; consistency is a property of the whole data set.

TASK 4 – Apply the Candidate Elimination Algorithm **Marks: 6**

(Irr = Irregular, Pas = Passive in the table below.) **Initialisation.** $S_0 = \{\langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle\}$, $G_0 = \{\langle ?, ?, ?, ?, ?, ? \rangle\}$.

Step	Example	Label	Action taken	S	G	Reason
0	–	–	Initialise	{⟨∅,∅,∅,∅,∅,∅⟩}	{⟨?,?,?,?,?,?⟩}	Most specific and most general.
1	E1	+	Generalise S minimally; keep G	{⟨Low,Irr,Low,Weak,Pas,No⟩}	{⟨?,?,?,?,?,?⟩}	S_0 does not cover E1, so S_1 = E1. G_0 covers E1, nothing removed.
2	E2	+	Generalise S on PriorCGPA	{⟨Low,Irr,Low,?,Pas,No⟩}	{⟨?,?,?,?,?,?⟩}	E2 differs from S_1 only in PriorCGPA (Weak vs Strong) → "?".
3	E3	–	Keep S; specialise G	{⟨Low,Irr,Low,?,Pas,No⟩}	{⟨Low,?,?,?,?,?⟩, ⟨?,Irr,?,?,?,?⟩, ⟨?,?,Low,?,?,?⟩, ⟨?,?,?,?,Pas,?⟩, ⟨?,?,?,?,?,No⟩}	S_2 does not cover E3 (Attendance differs) so S is unchanged. G_2 covers E3, so specialise (see below).
4	E4	+	Remove from G; generalise S	{⟨Low,Irr,?,?,Pas,?⟩}	{⟨Low,?,?,?,?,?⟩, ⟨?,Irr,?,?,?,?⟩, ⟨?,?,?,?,Pas,?⟩}	G members not covering E4 removed; S_3 does not cover E4 so generalise.
5	E5	–	Keep S; specialise one G member	{⟨Low,Irr,?,?,Pas,?⟩}	{⟨Low,?,?,?,?,?⟩, ⟨?,?,?,?,Pas,?⟩}	Only ⟨?,Irr,…⟩ covers E5; its specialisations are pruned (see below).

Detail for the positive examples (E1, E2, E4)

- **E1:** $S_0 = \langle \emptyset \ldots \rangle$ cannot cover E1, so the minimal generalisation is E1 itself. $G_0 = \langle ? \ldots \rangle$ covers E1, so no member of G is removed.
- **E2:** S_1 does not cover E2. The only mismatching constraint is PriorCGPA (Weak ≠ Strong), so it becomes "?". G unchanged because $\langle ? \ldots \rangle$ covers E2.
- **E4 (Low, Irregular, High, Weak, Passive, Yes):** members of G_3 that do **not** cover E4 are removed: ⟨?,?,Low,?,?,?⟩ (StudyHours = High ≠ Low) and ⟨?,?,?,?,?,No⟩ (Mentoring = Yes ≠ No). S_3 = ⟨Low,Irr,Low,?,Passive,No⟩ does not cover E4, so StudyHours becomes "?" and Mentoring becomes "?" → S_4 = ⟨Low,Irr,?,?,Passive,?⟩, which is still ≤ every member of G_4.

Detail for the negative example E3 (High, Regular, High, Strong, Active, Yes)

$G_2 = \{\langle ?,?,?,?,?,? \rangle\}$ covers E3, so it is replaced by its minimal specialisations: for each attribute, every value different from E3's value. A specialisation is kept only if it is more general than (or equal to) S_2 = ⟨Low,Irr,Low,?,Passive,No⟩.

Attribute specialised	Candidates	Kept?	Reason
Attendance	Low, Medium	Low kept; Medium rejected	Medium is not ≥ S_2 (S_2 requires Low).
Assignments	Irregular	Kept	E3 value is Regular; Irregular ≥ S_2.
StudyHours	Low	Kept	E3 value is High; Low ≥ S_2.
PriorCGPA	Weak, Strong	Both rejected	S_2 has "?" for PriorCGPA, so no specific value is ≥ S_2.
LMS	Passive	Kept	E3 value is Active.
Mentoring	No	Kept	E3 value is Yes.

The five kept hypotheses are mutually incomparable, so none is removed for being less general than another member of G.

Detail for the negative example E5 (Medium, Irregular, High, Strong, Active, No)

S_4 does not cover E5 (Attendance Low ≠ Medium), so S is unchanged. In G_4, ⟨Low,?,?,?,?,?⟩ and ⟨?,?,?,?,Passive,?⟩ do not cover E5 (Attendance Low, LMS Passive) and stay. ⟨?,Irr,?,?,?,?⟩ covers E5 and is specialised:

Specialisation	Decision	Reason
⟨Low,Irr,?,?,?,?⟩	Kept at first	≥ S_4, but later removed: less general than ⟨Low,?,?,?,?,?⟩ in G.
⟨High,Irr,?,?,?,?⟩	Rejected	Not ≥ S_4 (S_4 requires Low).
⟨?,Irr,Low,?,?,?⟩	Rejected	Not ≥ S_4 (S_4 has ? for StudyHours).
⟨?,Irr,?,Weak,?,?⟩ and ⟨?,Irr,?,?,?,Yes⟩	Rejected	Not ≥ S_4.
⟨?,Irr,?,?,Passive,?⟩	Kept at first	≥ S_4, but later removed: less general than ⟨?,?,?,?,Passive,?⟩ in G.

Final boundaries

S_5 = {⟨Low, Irregular, ?, ?, Passive, ?⟩} and **G_5 = {⟨Low, ?, ?, ?, ?, ?⟩, ⟨?, ?, ?, ?, Passive, ?⟩}**. S_5 and G_5 do not converge, so the version space is not a single hypothesis.

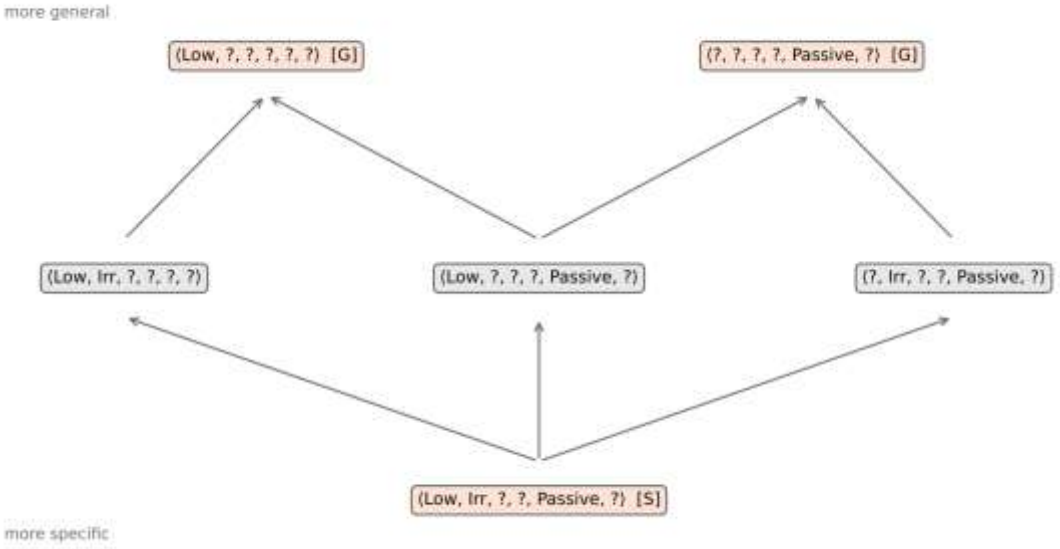
TASK 5 – Analyse Version Space and Inductive Bias **Marks: 4**

1. Version space

The version space contains every hypothesis between S_5 and G_5 (≥ S_5 and ≤ some member of G_5). Enumerating the generalisations of S_5 = ⟨Low, Irr, ?, ?, Passive, ?⟩ obtained by replacing constraints with "?" and keeping only those ≤ ⟨Low,?,?,?,?,?⟩ or ⟨?,?,?,?,Passive,?⟩ gives **6 hypotheses**:

Label	Hypothesis	Position
h1	⟨Low, Irregular, ?, ?, Passive, ?⟩	S boundary (most specific)
h2	⟨Low, Irregular, ?, ?, ?, ?⟩	between
h3	⟨Low, ?, ?, ?, Passive, ?⟩	between
h4	⟨?, Irregular, ?, ?, Passive, ?⟩	between
h5	⟨Low, ?, ?, ?, ?, ?⟩	G boundary
h6	⟨?, ?, ?, ?, Passive, ?⟩	G boundary

Figure 1. General-to-specific lattice of the version space (6 hypotheses)



Size of the version space = 6. The hypothesis ⟨?, Irregular, ?, ?, ?, ?⟩ is not in it because it covers the negative example **E5** (Medium, Irregular, …); it was removed from G_5 when E5 arrived. It is more general than S_5, but it is not consistent with the data.

2. Classification and ambiguity

Query	h1	h2	h3	h4	h5	h6	Yes : No	Verdict
Q1 = ⟨Low, Regular, High, Weak, Passive, No⟩	No	No	Yes	No	Yes	Yes	3 : 3	**Ambiguous**
Q2 = ⟨Low, Irregular, Low, Strong, Passive, Yes⟩	Yes	Yes	Yes	Yes	Yes	Yes	6 : 0	Positive (unanimous)

Q1 is ambiguous: it is Low attendance and Passive on LMS (covered by h3, h5, h6) but Regular on assignments (not covered by h1, h2, h4). Q2 satisfies every constraint that all six hypotheses share, so it is classified at-risk with certainty. The most informative next query is **Q1 itself (or any ⟨Low, Regular, ·, ·, Passive, ·⟩ student)**: it splits the version space 3 : 3, so either answer eliminates exactly half of it (1 bit). A query such as ⟨Medium, Irregular, Low, Weak, Passive, No⟩ splits it 2 : 4 and is less informative.

3. Four concepts

Concept	Meaning	Example from this problem
Training data	The labelled examples observed	E1–E5 with their AtRisk labels
Hypothesis space H	All hypotheses the learner is able to represent	973 semantically distinct conjunctions
Version space	Hypotheses in H consistent with all the training data	The 6 hypotheses h1–h6 after E5
Inductive bias	Assumptions that let the learner generalise beyond the data	Target is a conjunction of attribute constraints ($c \in H$)

4. Inductive bias

Bias of Candidate Elimination here: the true concept can be written as a single conjunction of the six attribute constraints (it is in H) and the data are noise-free. Because of this, the learner can classify instances it has never seen.

Effect of a new positive example ⟨High, Regular, High, Strong, Passive, No⟩. S_5 = ⟨Low,Irr,?,?,Passive,?⟩ does not cover it, so Attendance and Assignments are generalised to "?": **S_6 = ⟨?, ?, ?, ?, Passive, ?⟩**. In G, ⟨Low,?,?,?,?,?⟩ does not cover it and is removed; ⟨?,?,?,?,Passive,?⟩ stays. So $S_6 = G_6$ = ⟨?, ?, ?, ?, Passive, ?⟩ and the version space shrinks from 6 hypotheses to **1**. The learner is now forced to claim "AtRisk if and only if LMS = Passive", so every one of the 48 Passive instances is labelled at-risk and every Active instance safe, a strong commitment made on very little evidence; and a true concept such as "Low attendance OR Passive" cannot be represented at all.

Unbiased learner (H = power set of X). The number of possible concepts is $2^{96} \approx 7.92 \times 10^{28}$. After E1–E5, $2^{91} \approx 2.48 \times 10^{27}$ of them remain consistent. For any unseen instance exactly half of the remaining concepts say Yes and half say No, so the learner has no basis to classify it. Its S boundary is just the disjunction of the positive examples and its G boundary is the negation of the negative examples: it can only memorise. This is why **learning cannot happen without an inductive bias**.

TASK 6 – Construct the Decision Tree **Marks: 6**

1. Entropy of the full training set (Dataset B)

Dataset B has 14 examples: 7 Yes and 7 No. $H(S) = -(7/14)\log_2(7/14) - (7/14)\log_2(7/14) = 0.5 + 0.5 =$ **1.0000**.

2. Root split – all four attributes

Weighted entropy = $\Sigma\ (|S_v|/|S|) \cdot H(S_v)$; Gain = H(S) − weighted entropy.

Attribute	Value	Size	Yes / No	H(subset)	Weighted entropy	Gain
Attendance	Low	5	4 / 1	0.7219	(5/14)(0.7219) + (5/14)(0.9710) + (4/14)(0.8113) = 0.8364	**0.1636**
	Medium	5	2 / 3	0.9710		
	High	4	1 / 3	0.8113		
Assignments	Irregular	8	5 / 3	0.9544	(8/14)(0.9544) + (6/14)(0.9183) = 0.9389	**0.0611**
	Regular	6	2 / 4	0.9183		
LMS	Passive	7	5 / 2	0.8631	(7/14)(0.8631) + (7/14)(0.8631) = 0.8631	**0.1369**
	Active	7	2 / 5	0.8631		
PriorCGPA	Strong	7	1 / 6	0.5917	(7/14)(0.5917) + (7/14)(0.5917) = 0.5917	**0.4083**
	Weak	7	6 / 1	0.5917		

Root attribute: PriorCGPA (highest gain, 0.4083). Branches: Strong (1 Yes, 6 No) and Weak (6 Yes, 1 No); both are impure.

3. Impure branches

Stopping criteria: (i) the node is pure (all one class) → leaf with that class; (ii) no attributes remain → leaf with the majority class; (iii) an empty subset → majority class of the parent. **Tie-breaking rule:** if two attributes have equal gain, choose the one that comes first in the order Attendance, Assignments, LMS, PriorCGPA; if a leaf has equal class counts, label it **Yes** (the costlier error is a missed at-risk student).

Branch PriorCGPA = Strong (7 examples: 1 Yes, 6 No; H = 0.5917)

Attribute	Subsets (Yes/No)	Weighted entropy	Gain
Attendance	High 0/2 (0); Low 1/1 (1.0000); Medium 0/3 (0)	(2/7)(1.0000) = 0.2857	**0.3060**
Assignments	Irregular 1/3 (0.8113); Regular 0/3 (0)	(4/7)(0.8113) = 0.4636	0.1281
LMS	Passive 1/2 (0.9183); Active 0/4 (0)	(3/7)(0.9183) = 0.3935	0.1981

Select **Attendance**. High and Medium are pure → leaves "No". Low (E2 Yes, E4 No; 1 Yes, 1 No; H = 1.0000) is impure.

Branch PriorCGPA = Weak (7 examples: 6 Yes, 1 No; H = 0.5917)

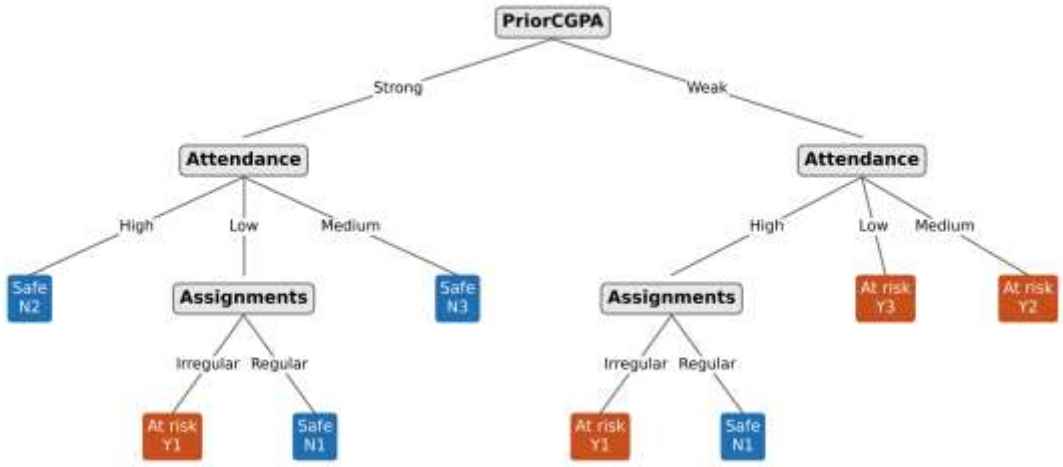
Attribute	Subsets (Yes/No)	Weighted entropy	Gain
Attendance	High 1/1 (1.0000); Low 3/0 (0); Medium 2/0 (0)	(2/7)(1.0000) = 0.2857	**0.3060**
Assignments	Irregular 4/0 (0); Regular 2/1 (0.9183)	(3/7)(0.9183) = 0.3935	0.1981
LMS	Passive 4/0 (0); Active 2/1 (0.9183)	(3/7)(0.9183) = 0.3935	0.1981

Select **Attendance**. Low and Medium are pure → leaves "Yes". High (1 Yes, 1 No) is impure.

Sub-branches (Strong, Low) and (Weak, High) – two examples each, one Yes and one No, H = 1.0000. Candidates left: Assignments and LMS. In both nodes Assignments gives two pure subsets (gain = 1.0000) and so does LMS (gain = 1.0000): a **tie**. By the tie-breaking rule Assignments is selected. Both resulting branches are pure, so the tree stops.

4. Final tree, rules and statistics

Figure 2. ID3 tree on Dataset B (Y = at risk count, N = safe count in each leaf)



IF–THEN rules (8) :

- R1: IF PriorCGPA = Strong AND Attendance = High THEN AtRisk = No
- R2: IF PriorCGPA = Strong AND Attendance = Medium THEN AtRisk = No
- R3: IF PriorCGPA = Strong AND Attendance = Low AND Assignments = Irregular THEN AtRisk = Yes
- R4: IF PriorCGPA = Strong AND Attendance = Low AND Assignments = Regular THEN AtRisk = No
- R5: IF PriorCGPA = Weak AND Attendance = Low THEN AtRisk = Yes
- R6: IF PriorCGPA = Weak AND Attendance = Medium THEN AtRisk = Yes
- R7: IF PriorCGPA = Weak AND Attendance = High AND Assignments = Irregular THEN AtRisk = Yes
- R8: IF PriorCGPA = Weak AND Attendance = High AND Assignments = Regular THEN AtRisk = No

Depth	Total nodes	Internal nodes	Leaves	Training accuracy
3	13	5	8	14 / 14 = 100 %

TASK 7 – Analyse Heuristic Search in Decision Tree Learning Marks: 3

1. ID3 as a search problem

Component	Description	In my tree
Start state	The empty tree: a single leaf covering all 14 examples (majority class)	Root leaf: 7 Yes / 7 No
Operator	Replace a leaf by a decision node on one unused attribute, creating one child per value	PriorCGPA at root; Attendance twice; Assignments twice
Heuristic	Information gain (highest gain wins; ties by attribute order)	Gain 0.4083 at root, 0.3060 at level 1, 1.0000 at level 2
Termination	Node pure, or no attributes left, or no examples	All 8 leaves are pure

Decision node	Candidate attributes evaluated	Gain computations
Root (14 examples)	Attendance, Assignments, LMS, PriorCGPA	4
PriorCGPA = Strong (7 ex.)	Attendance, Assignments, LMS	3
PriorCGPA = Weak (7 ex.)	Attendance, Assignments, LMS	3
Strong, Low (2 ex.)	Assignments, LMS	2
Weak, High (2 ex.)	Assignments, LMS	2
Total		**14**

Size of the search space. The instance space for Dataset B has 24 points, so there are 2^{24} = 16,777,216 possible Boolean concepts, and every one of them can be written as some decision tree. ID3 therefore searches a space that contains **every** concept (very expressive, almost no representational bias) but explores only one path through it, using only 14 gain computations. In contrast, conjunctive concept learning searches only 973 hypotheses (Task 3). The tree learner's bias is thus a **preference bias** (prefer shorter trees with high-gain attributes near the root) rather than a restriction bias; the conjunctive learner has a strong restriction bias and cannot represent disjunctions.

2. Information gain versus gain ratio at the root

Split information $SI = -\Sigma\ (|S_v|/|S|)\ \log_2(|S_v|/|S|)$; GainRatio = Gain / SI.

Attribute	Subset sizes	Split information	Gain	Gain ratio
PriorCGPA	7, 7	1.0000	0.4083	0.4083 / 1.0000 = **0.4083**
Attendance	5, 5, 4	2(5/14)(1.4854) + (4/14)(1.8074) = 0.5305 + 0.5305 + 0.5164 = 1.5774	0.1636	0.1636 / 1.5774 = **0.1037**
Assignments	8, 6	0.9852	0.0611	0.0620
LMS	7, 7	1.0000	0.1369	0.1369

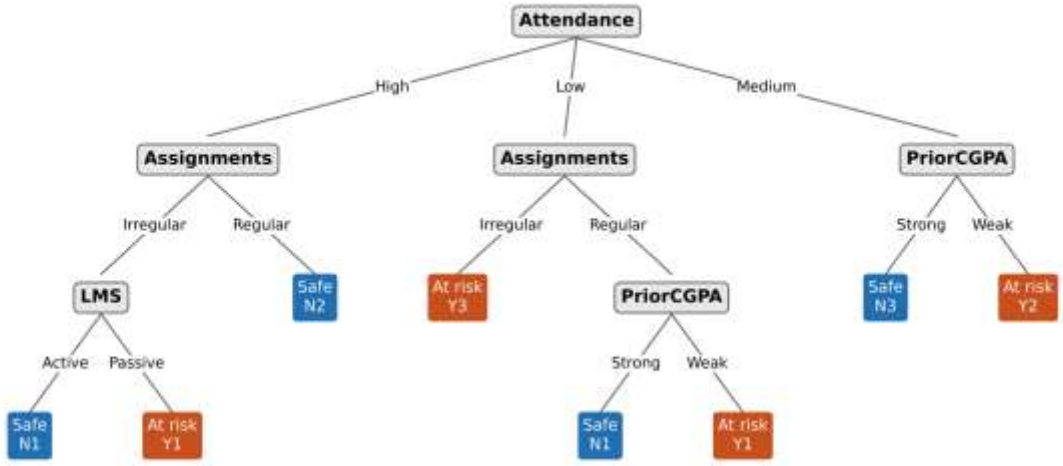
Both measures choose PriorCGPA. Gain ratio divides by split information, which grows with the number of branches and how evenly they split the data, so many-valued attributes are penalised. **StudentID:** a unique identifier splits the 14 examples into 14 pure singletons, each with entropy

0, so weighted entropy = 0 and **Gain = 1.0000 − 0 = 1.0000**, higher than PriorCGPA's 0.4083. Choosing it would be a poor decision because the tree would memorise students and could not classify anyone new. Its split information is $\log_2 14 = 3.8074$, so its gain ratio is 1.0000 / 3.8074 = 0.2626, which is lower than PriorCGPA's 0.4083, so gain ratio would reject it. **I adopt information gain on Dataset B** (all attributes have 2 or 3 values, no identifiers are used, and both measures agree), and I would switch to gain ratio if multi-valued or identifier-like attributes were added.

3. Greedy and local decision making

Alternative tree with **Attendance** at the root (same stopping rules). Gains inside each branch: Attendance = High (1 Yes, 3 No): Assignments, LMS and PriorCGPA all give 0.3113 (three-way tie → Assignments); then Irregular (1/1) splits on LMS (tie with PriorCGPA, LMS first). Attendance = Low (4 Yes, 1 No): Assignments 0.3219, LMS 0.1709, PriorCGPA 0.3219 (tie → Assignments); then Regular (1/1) splits on PriorCGPA (LMS gain is 0). Attendance = Medium (2 Yes, 3 No): PriorCGPA 0.9710 (pure split), LMS 0.4200, Assignments 0.0200.

Figure 3. Alternative tree with Attendance forced at the root



Measure	Greedy tree (PriorCGPA root)	Alternative (Attendance root)
Leaves	8	8
Total nodes	13	14
Depth	3	3
Training accuracy	14 / 14 = 100 %	14 / 14 = 100 %
Test accuracy on T	3 / 6 = 50 %	2 / 6 = 33.33 %

The greedy choice produced a slightly **smaller** tree (13 against 14 nodes, one fewer decision node) with the same leaves and depth, and a marginally better test result; the difference on six test cases is not statistically meaningful. ID3 **never backtracks** because it commits to the attribute with the best immediate gain, keeps no record of alternatives and refines each branch

independently; re-examining earlier choices would need to search the huge space of trees, which is exactly what the heuristic avoids.

Advantages of heuristic search: (1) it is cheap – only 14 gain computations built a complete consistent tree, against exploring many of the 2^{24} concepts; (2) the gain heuristic gave an interpretable tree with the single most informative attribute (PriorCGPA) at the root, where gain was 0.4083, far above the next best 0.1636. **Limitations:** (1) it is greedy and locally optimal: a different root (Attendance) gives a different, only slightly larger tree, so there is no guarantee of the smallest tree; (2) it overfits: it kept splitting until each leaf was pure, so leaves hold only 1–3 students and the tree scored just 50 % on T.

TASK 8 – Analyse Overfitting and Underfitting **Marks: 3**

1. Diagnosis

M0 predicts the majority class; the classes tie 7 : 7, so by the tie rule it predicts **Yes** for everyone. Test set T has labels No, No, Yes, Yes, No, Yes (3 Yes, 3 No).

Model	Depth	Nodes / leaves	Training accuracy	Test accuracy	Gap	Diagnosis
M0 – no splits	0	1 / 1	7/14 = 50.00 %	3/6 = 50.00 %	0	**Underfits**
M1 – depth 1	1	3 / 2	12/14 = 85.71 %	5/6 = 83.33 %	2.4 pts	**Best generalisation**
M2 – full tree	3	13 / 8	14/14 = 100 %	3/6 = 50.00 %	50 pts	**Overfits**

Test predictions of M2: T1 Yes (actual No ✗), T2 Yes (No ✗), T3 No (Yes ✗), T4 Yes ✓, T5 No ✓, T6 Yes ✓ → 3/6. M1 splits only on PriorCGPA (Strong → No, Weak → Yes) and errs only on T2 → 5/6. M0 has low accuracy on both sets, which signals underfitting (high bias). M2 reaches 100 % on training but drops 50 points on test and has 13 nodes for 14 examples, which signals overfitting. The best generalisation is at **depth 1 (M1)**.

2. Causes

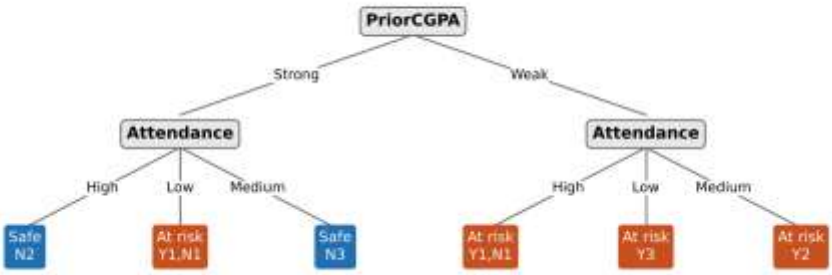
- **Very small sample:** only 14 training students, so any pattern, including chance ones, can be fitted.
- **Coverage:** B covers 14 of 24 combinations (58.3 %); the test students come from combinations the tree has never seen, and the deeper splits (Attendance, Assignments) were learned from only 1–3 students.
- **Tiny leaves:** 4 of the 8 leaves contain one student, so one atypical student sets a rule.
- **Possible noise:** the full tree misclassifies T1, T2 and T3. T2 (Weak but not at risk) contradicts the strong Weak → at-risk pattern in B and may be noise, while T1 and T3 fall into leaves built from a single student each (students 2 and 12 of Dataset B), which is exactly how one atypical record turns into a rule.

3. Control methods applied

Method	Setting	Tree	Train acc.	Test acc.
Maximum depth	depth = 1	M1: 3 nodes, 2 leaves	85.71 %	83.33 %
Maximum depth	depth = 2	9 nodes, 6 leaves (same tree as min_samples_split = 4)	85.71 %	50.00 %
Minimum samples per split	min_samples_split = 4	Nodes with fewer than 4 students become leaves: 9 nodes, 6 leaves	85.71 %	50.00 %
Minimum information gain	threshold 0.35	Level-1 gains are 0.3060 < 0.35, so splitting stops: same as M1	85.71 %	83.33 %
Cross-validation (leave-one-out, 20 records)	choose depth	Depth 0: 50 %, depth 1: 85 %, depth 2: 85 %, depth 3: 55 %	–	–

Pruned tree for min_samples_split = 4 (hand-derived): the root (14) and the two PriorCGPA branches (7 each) are split; the Attendance nodes have 2, 2, 3 students (< 4), so they become leaves. The tied leaves (Strong, Low: 1 Yes / 1 No) and (Weak, High: 1 Yes / 1 No) are resolved with the rule **tie → Yes**. Rules: Strong & High → No; Strong & Medium → No; Strong & Low → Yes (tie); Weak & any attendance → Yes.

Figure 5. Tree with min_samples_split = 4 (tied leaves resolved to At risk)



Training accuracy: errors are students 4 and 12 of Dataset B (one in each tied leaf) → 12/14 = 85.71 %. Test accuracy 3/6 = 50 %: this tree predicts Yes for every test student (the Weak branch and both tied leaves say Yes), so the three safe students T1, T2 and T5 are flagged. Its recall on T is 100 % but its precision is only 50 %. Limiting the depth to 2 therefore does not help; the benefit comes from stopping at depth 1.

4. Final complexity and reliability

I choose **maximum depth 1** (M1; equivalently a minimum-gain threshold of 0.35): it gives the best test accuracy (83.33 %), the best leave-one-out accuracy (85 %), recall 100 % on T, and a rule an advisor can quote in one line ("weak prior CGPA → at risk"). **Reliability:** with six test examples, one example moves accuracy by 16.7 points; the 95 % Wilson interval for 5/6 is about 44 % to 97 %, so the conclusion "M1 beats M2" is suggestive, not proven. To improve the evaluation I would use repeated stratified k-fold cross-validation on all labelled data, report recall and confidence intervals, and replace T with a much larger held-out set (the 2,000 records of Task 11).

TASK 9 – Implement and Test the ML Techniques **Marks: 5**

Everything was implemented from scratch in Python 3 (standard library only; NumPy/matplotlib were used only for the figures). The full source is in Appendix A. No library routine is used for Candidate Elimination or for the decision tree; scikit-learn was not needed.

Component	Implementation (file ml.py)
Concept-learning representation	Hypotheses are tuples with "?" and "0" (∅). covers(h, x) and more_general_eq(h1, h2) implement matching and the ≥g order.
Candidate Elimination	S and G kept as Python sets; min_gen() generalises S, min_spec() specialises G; non-maximal / non-minimal members are pruned; trace is printed after every example; an empty S or G is detected and reported.
Decision tree	entropy(), gain(), split_info(), recursive id3() with stopping rules (pure, no attributes, max_depth, min_split, min_gain), deterministic tie-break by attribute order, predict() with unseen-value policy, rules() and show() for printing.
Reproducibility	random.seed(42); comments in code; run with "python run_all.py" and "python tests.py".

README (summary)

```
ITA06 Assignment 1 - Early-warning system (Candidate Elimination + ID3)
Files : ml.py (algorithms + datasets A, B, T)  run_all.py (logs)  tests.py (TC1-TC8)
        exp.py / exp2.py / exp3.py (experiments, Dataset C generator)  figs.py (plots)
Run   : python3 run_all.py   -> S/G after each example, entropy/gain per node, tree, rules, T predictions
        python3 tests.py     -> test-case table, prints PASS/FAIL
Needs : Python 3.9+ (standard library); matplotlib only for figs.py. Seed fixed at 42.
Policy: tie in gain -> first attribute in order Attendance, Assignments, LMS, PriorCGPA.
        tie in leaf classes -> "Yes".  unseen attribute value -> majority class of the current node.
```

Test cases

Test	Input	Expected behaviour	Observed output	Pass/Fail
TC1	Candidate Elimination on Dataset A	S and G after each example equal the hand trace of Task 4; final S_5, G_5	S_5 = {⟨Low,Irr,?,?,Pas,?⟩}, G_5 = {⟨Low,?,?,?,?,?⟩, ⟨?,?,?,?,Pas,?⟩}; all 5 steps identical to Task 4	PASS
TC2	Entropy of 5 Yes (pure) and of 7 Yes / 7 No	0 and 1	0.0000 and 1.0000	PASS
TC3	Gain of the 4 attributes at root of B	Match Task 6 table	Attendance 0.1636, Assignments 0.0611, LMS 0.1369, PriorCGPA 0.4083	PASS
TC4	Tree built from B, 14 training examples	All 14 correct; 8 rules as in Task 6	Training accuracy 1.0000; depth 3, 13 nodes, 8 leaves, 8 rules	PASS
TC5	Predictions on Test Set T	Match Task 8 (3/6)	Yes, Yes, No, Yes, No, Yes; accuracy 0.5000	PASS
TC6	Dataset A + E6 (contradicts E1)	Report empty version space, no crash	"VERSION SPACE EMPTY at example 6" printed; function returns normally	PASS
TC7	Unseen value Attendance = "Very Low"	Defined, justified policy	Missing branch → majority class of the current node; ⟨Very Low, Irr, Pas, Weak⟩ → Yes. Justification: it uses the best evidence available at that node without crashing and without inventing a branch.	PASS
TC8	Tie in gain (Assignments = LMS = 1.0000 at depth-2 nodes)	Deterministic choice	Assignments chosen at both nodes in every run (attribute order rule)	PASS

Console log (run_all.py)

```
=== Candidate Elimination, Dataset A (S and G after every example) ===
E1 (+)
  S = {<Low,Irr,Low,Weak,Pas,No>}
  G = {<?,?,?,?,?,?>}
E2 (+)
  S = {<Low,Irr,Low,?,Pas,No>}
  G = {<?,?,?,?,?,?>}
E3 (-)
  S = {<Low,Irr,Low,?,Pas,No>}
  G = {<?,?,?,?,?,No>, <?,?,?,?,Pas,?>, <?,?,Low,?,?,?>, <?,Irr,?,?,?,?>, <Low,?,?,?,?,?>}
E4 (+)
  S = {<Low,Irr,?,?,Pas,?>}
  G = {<?,?,?,?,Pas,?>, <?,Irr,?,?,?,?>, <Low,?,?,?,?,?>}
E5 (-)
  S = {<Low,Irr,?,?,Pas,?>}
  G = {<?,?,?,?,Pas,?>, <Low,?,?,?,?,?>}


=== ID3 on Dataset B: gain per attribute per node ===
H(S) = 1.0000
depth 0, n=14, class counts {'Yes': 7, 'No': 7}
   gain(Attendance) = 0.1636
   gain(Assignments) = 0.0611
   gain(LMS) = 0.1369
   gain(PriorCGPA) = 0.4083
   -> split on PriorCGPA
depth 1, n=7, class counts {'Yes': 1, 'No': 6}
   gain(Attendance) = 0.3060
   gain(Assignments) = 0.1281
   gain(LMS) = 0.1981
   -> split on Attendance
depth 2, n=2, class counts {'Yes': 1, 'No': 1}
   gain(Assignments) = 1.0000
   gain(LMS) = 1.0000
   -> split on Assignments
depth 1, n=7, class counts {'Yes': 6, 'No': 1}
   gain(Attendance) = 0.3060
   gain(Assignments) = 0.1981
   gain(LMS) = 0.1981
   -> split on Attendance
depth 2, n=2, class counts {'Yes': 1, 'No': 1}
```

```
    gain(Assignments) = 1.0000
    gain(LMS) = 1.0000
    -> split on Assignments


=== Printed tree ===
PriorCGPA = Strong
    Attendance = High
        -> No {'No': 2}
    Attendance = Low
        Assignments = Irregular
            -> Yes {'Yes': 1}
        Assignments = Regular
            -> No {'No': 1}
    Attendance = Medium
        -> No {'No': 3}
PriorCGPA = Weak
    Attendance = High
        Assignments = Irregular
            -> Yes {'Yes': 1}
        Assignments = Regular
            -> No {'No': 1}
    Attendance = Low
        -> Yes {'Yes': 3}
    Attendance = Medium
        -> Yes {'Yes': 2}


=== Rules ===
IF PriorCGPA = Strong AND Attendance = High THEN AtRisk = No   [{'No': 2}]
IF PriorCGPA = Strong AND Attendance = Low AND Assignments = Irregular THEN AtRisk = Yes   [{'Yes': 1}]
IF PriorCGPA = Strong AND Attendance = Low AND Assignments = Regular THEN AtRisk = No   [{'No': 1}]
IF PriorCGPA = Strong AND Attendance = Medium THEN AtRisk = No   [{'No': 3}]
IF PriorCGPA = Weak AND Attendance = High AND Assignments = Irregular THEN AtRisk = Yes   [{'Yes': 1}]
IF PriorCGPA = Weak AND Attendance = High AND Assignments = Regular THEN AtRisk = No   [{'No': 1}]
IF PriorCGPA = Weak AND Attendance = Low THEN AtRisk = Yes   [{'Yes': 3}]
IF PriorCGPA = Weak AND Attendance = Medium THEN AtRisk = Yes   [{'Yes': 2}]


Stats (depth,nodes,leaves) = (3, 13, 8)  training accuracy = 1.0000


=== Predictions on Test Set T ===
T1 ('Low', 'Irregular', 'Active', 'Strong') actual=No predicted=Yes
T2 ('Medium', 'Regular', 'Active', 'Weak') actual=No predicted=Yes
T3 ('High', 'Regular', 'Passive', 'Weak') actual=Yes predicted=No
T4 ('Medium', 'Irregular', 'Active', 'Weak') actual=Yes predicted=Yes
T5 ('Low', 'Regular', 'Passive', 'Strong') actual=No predicted=No
T6 ('High', 'Irregular', 'Active', 'Weak') actual=Yes predicted=Yes
test accuracy = 0.5000
```

TASK 10 – Experimental Analysis, Theory vs Experiment, Recommendation Marks: 6

10A. Experimental analysis

Dataset C (scalability). A documented synthetic generator with a configurable noise rate, written in Python (seed 42). Each of 600 records draws the six attributes of Dataset A uniformly at random. The rule for AtRisk = Yes is: (PriorCGPA = Weak **and not** (Attendance = High **and** Assignments = Regular)) **or** (PriorCGPA = Strong **and** Attendance = Low **and** Assignments = Irregular); StudyHours and Mentoring are irrelevant. This rule reproduces all 14 labels of Dataset B. Label noise (0, 5, 10 %) is added by flipping that proportion of the **training** labels; validation labels are kept clean. Split: 420 training / 180 validation records.

1. Dataset characteristics

Dataset	Examples	Attributes	Class balance (Yes / No)	Cardinalities	Instance space	Coverage
A	5	6	3 / 2	3,2,2,2,2,2	96	5.21 %
B	14	4	7 / 7	3,2,2,2	24	58.33 %
T	6	4	3 / 3	3,2,2,2	24	25.00 % (disjoint from B)
C	600	6	270 / 330	3,2,2,2,2,2	96	100 % (96 distinct points)

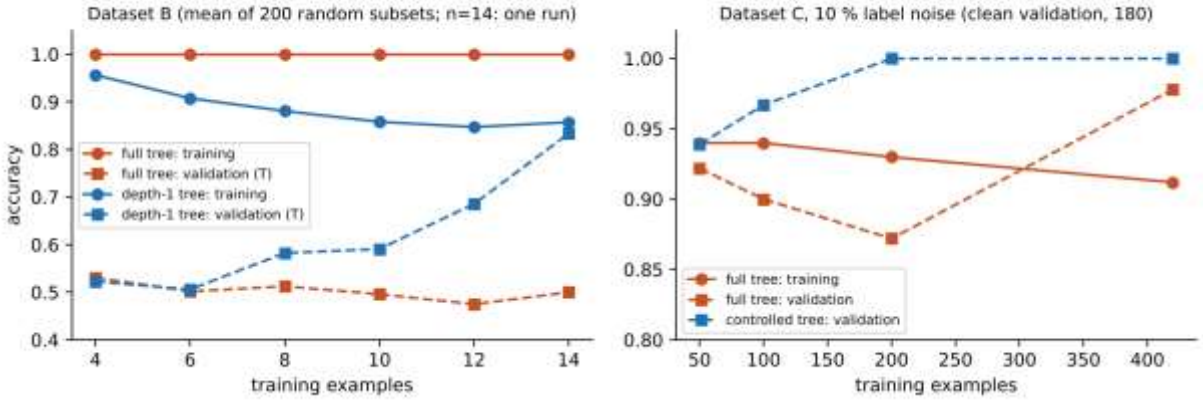
2. Performance on T and cross-validation (positive class = At risk)

Evaluation	Model	TP	FN	FP	TN	Accuracy	Precision	Recall	F1
Test set T (6)	Full tree	2	1	2	1	0.5000	0.5000	0.6667	0.5714
Test set T (6)	Depth-1 tree	3	0	1	2	0.8333	0.7500	1.0000	0.8571
Leave-one-out, B+T (20)	Full tree	6	4	5	5	0.5500	0.5455	0.6000	0.5714
Leave-one-out, B+T (20)	Depth-1 tree	9	1	2	8	0.8500	0.8182	0.9000	0.8571
Leave-one-out, B+T (20)	min_samples_split = 4	6	4	2	8	0.7000	0.7500	0.6000	0.6667

The confusion matrix of the leave-one-out depth-1 tree is [[TP 9, FN 1], [FP 2, TN 8]]. The fully grown tree misses 4 of the 10 at-risk students (recall 0.60), far below the 0.90 requirement.

3. Effect of training-set size (learning curve)

Figure 6. Learning curves

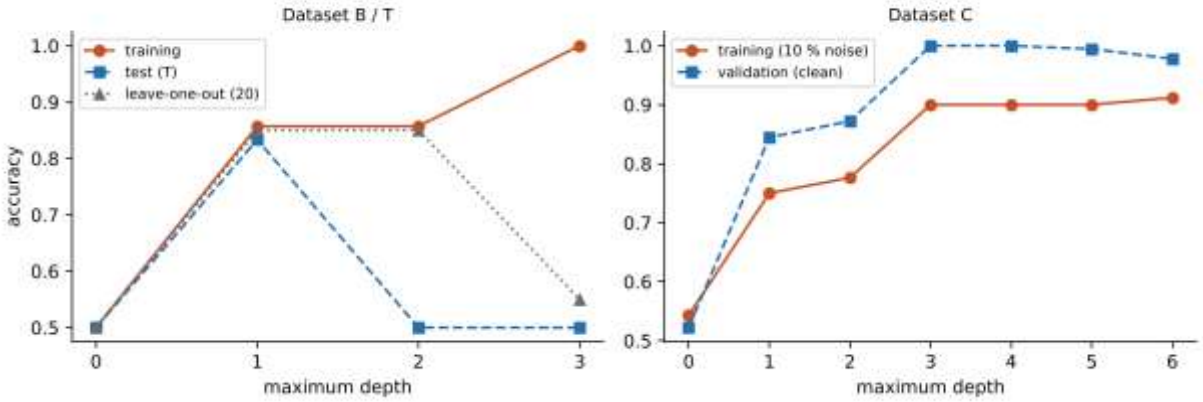


Training size (Dataset B subset)	4	6	8	10	12	14
Full tree – training accuracy	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Full tree – validation on T	0.5308	0.5017	0.5125	0.4958	0.4750	0.5000
Depth-1 tree – training accuracy	0.9563	0.9075	0.8806	0.8580	0.8471	0.8571
Depth-1 tree – validation on T	0.5225	0.5058	0.5825	0.5908	0.6858	0.8333

(Means of 200 random subsets for n < 14.) The full tree always memorises its training set (100 %) but validation stays at about 50 %: adding data from B alone does not help because T disagrees with the pattern of B. The depth-1 tree starts with high training accuracy that decreases and meets validation accuracy as data grow (0.857 vs 0.833), the classic converging learning curve. On Dataset C with 10 % noise the controlled tree reaches 0.939, 0.967, 1.000, 1.000 validation accuracy for 50, 100, 200, 420 records, while the full tree reaches 0.922, 0.900, 0.872, 0.978; without noise both reach 1.000 at every size.

4. Effect of model complexity (maximum depth)

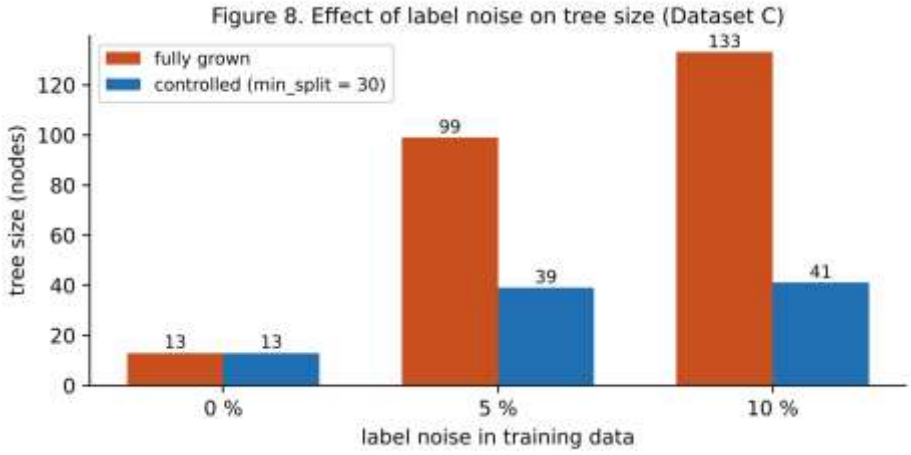
Figure 7. Accuracy against model complexity



Dataset	Depth	Nodes	Leaves	Training acc.	Validation / test acc.
B / T	0	1	1	0.5000	0.5000
B / T	1	3	2	0.8571	0.8333
B / T	2	9	6	0.8571	0.5000
B / T (full)	3	13	8	1.0000	0.5000
C (10 % noise)	0	1	1	0.5429	0.5222
C (10 % noise)	1	3	2	0.7500	0.8444
C (10 % noise)	2	9	6	0.7762	0.8722
C (10 % noise)	3	21	12	0.9000	1.0000
C (10 % noise)	4	43	23	0.9000	1.0000
C (10 % noise)	5	77	40	0.9000	0.9944
C (10 % noise, full)	6	133	68	0.9119	0.9778

On B/T the validation score peaks at depth 1 and collapses at depth 2–3 (overfitting). On C the best depth is 3, where the tree has recovered the true rule; deeper trees only add nodes that fit the 10 % noise, so training accuracy rises slightly while validation falls.

5. Effect of label noise (Dataset C)



Training label noise	Tree	Nodes	Leaves	Training acc.	Validation acc. (clean)
0 %	Fully grown	13	8	1.0000	1.0000
0 %	Controlled (min_split = 30)	13	8	1.0000	1.0000
5 %	Fully grown	99	51	0.9548	0.9944
5 %	Controlled (min_split = 30)	39	21	0.9500	1.0000
10 %	Fully grown	133	68	0.9119	0.9778
10 %	Controlled (min_split = 30)	41	22	0.9000	1.0000

Noise inflates the fully grown tree from 13 to 133 nodes (10×) while accuracy falls; the controlled tree grows to only 41 nodes and keeps 100 % on clean data.

6. Candidate Elimination versus decision tree

Candidate Elimination on Dataset B (same four attributes, Yes = positive) – the version space **collapses at example 4** (Low, Regular, Active, Strong → No). After the first three positives S = ⟨Low, ?, ?, ?⟩, which covers this negative example, so S is removed and G becomes empty. The concept in B is not a single conjunction (at-risk students have Weak CGPA **or** Low attendance, etc.).

Aspect	Candidate Elimination	Decision tree (ID3)
Representable concepts	Single conjunction of constraints (973 hypotheses over A)	Any Boolean function (2^{24} over B)
Result on Dataset B	Version space empty after 4 examples – no classifier	Consistent 8-leaf tree, 100 % training accuracy
Noisy / contradictory data	One bad label empties the version space	Overfits unless pruned; handled by pruning and min-samples
Unseen values and combinations	Cannot classify beyond its fixed domains; ambiguous queries give split votes	Needs a back-off policy; otherwise classifies every path
Data requirement	Few examples if the concept is conjunctive and noise-free	More examples; best with hundreds of records
Interpretability	Very high (one rule), boundaries show uncertainty	High (rule set), depends on tree size
Output on this task	Ambiguity: Q1 voted 3 : 3	Hard class with leaf counts

10. Theory versus experimental behaviour

- **Overfitting of the full tree – supported.** Theory predicted 100 % training and poor test accuracy for the 8-leaf tree; observed 100 % and 50 % (LOO 55 %), with the depth-1 tree reaching 83–85 %. On Dataset C the gap appeared only with noise (full tree 133 nodes).
- **Collapse of the version space – supported.** Predicted for non-conjunctive or noisy data; observed on B at example 4, after E6 (step 6) and after E8.
- **Dataset characteristics.** Size: with 14–20 records, estimates are unstable (one test case = 16.7 points). Coverage: B covers 58 %, and the learning curve on B stays flat; C covers the whole 96-point space and reaches 100 %. Noise: 5–10 % noise multiplies tree size by 8–10× and reduces accuracy of the full tree (0.9778 on clean validation).
- **Inductive bias.** The conjunctive bias is too strong for B (collapse) while the tree's bias is weak enough to fit everything and therefore needs control, matching the strong-restriction versus weak-preference-bias argument of Task 7.
- **Heuristic selection.** Information gain and gain ratio picked the same root (PriorCGPA) on B; on Dataset C with 10 % noise they gave the same validation accuracy (0.9778) with trees of 133 and 132 nodes, so the choice mattered little with 2–3-valued attributes; the Attendance-rooted tree was one node larger and less accurate on T (33 % vs 50 %), showing mild sensitivity to the greedy root choice.
- **Model complexity.** Validation accuracy peaks at depth 1 on B and depth 3 on C and falls beyond, the expected U-shape in generalisation error.
- **Limitations and threats to validity.** Only 20 real labelled records; T disagrees partly with B, so six test examples give wide intervals (5/6 → about 44–97 %); Dataset C is synthetic with uniform attributes and a rule that I wrote (so a tree can recover it exactly); noise is random rather than advisor-specific; leave-one-out on 20 records has high variance; thresholds were not tuned on separate data.

Evidence-based recommendation

Recommend a controlled (pruned, depth/leaf-limited) decision tree with a recall-oriented flagging threshold, using Candidate Elimination only as a consistency checker.

Criterion	Candidate Elimination	Full tree	Controlled tree (recommended)
Accuracy / recall (LOO, 20 records)	No classifier after 4 examples of B	0.55 / 0.60	0.85 / 0.90
Recall on T	–	0.6667	1.0000
Model complexity	1 hypothesis	13 nodes, 8 leaves (133 on noisy C)	3 nodes (41 on noisy C)
Interpretability	Best	Good but long rules	Best of the trees: 1–3 conditions
Noise handling	Fails (empties)	Memorises noise	Tolerant
Unseen cases	Fixed domains	Needs a policy	Back-off to node majority
Data needs	Very small	Moderate	Moderate

The recommendation follows from measured evidence: the controlled tree improved leave-one-out recall from 0.60 to 0.90 and F1 from 0.57 to 0.86, and kept 100 % clean-validation accuracy at 10 % noise with one third of the nodes. **Candidate Elimination would be preferable** when the true concept is known to be a single conjunction, the labels are verified noise-free and data are scarce (fewer than about 20 examples), because it then yields a compact answer and a measure of uncertainty (the version space) that guides which student to label next.

TASK 11 – Re-evaluate and Adapt the Design **Marks: 8**

A. Impact analysis

Change	Original assumption violated	Affects	Consequence for concept learning	Consequence for decision trees
About 2,000 records, weekly arrivals	Small, static data	Data and evaluation	More examples make a contradiction more likely; the version space can only shrink and never recovers	Splits become reliable and deeper trees possible; need regular rebuilds and a proper validation set
5–10 % inconsistent labels	Noise-free labels	Data and algorithm	A single wrong label empties the version space (E6 does)	Full tree memorises wrong labels (N2 adds splits and leaves)
New values and combinations (Very Low)	Fixed attribute values	Hypothesis space and data	Fixed domains: "Very Low" is outside the hypothesis language, specialisations cannot enumerate it	A node has no branch for it unless a policy is defined
At least 90 % recall plus explanation	Accuracy-only evaluation	Evaluation	No classifier remains after collapse; no recall can be measured	Must tune leaf threshold or class weights; rules give the explanation
E7 and E8 show disjunction	Conjunctive target concept	Hypothesis space	S and G cannot cover Passive and Active positives (E8 empties the space)	Trees represent disjunctions of conjunctions naturally

N2 versus E6. E6 has exactly the same attribute values as E1 but the opposite label: no hypothesis can be consistent with both, so a method that assumes consistent data (Candidate Elimination) fails immediately, while a noise-tolerant method treats the pair as a 1 : 1 conflict

and lets the majority or leaf probability decide (a tie goes to Yes). N2 (High, Regular, Active, Strong → Yes) does not contradict any single record, but it contradicts the rule learned from the neighbouring students (Strong with high attendance is safe). A consistency-assuming tree adds an extra split on LMS to isolate N2 and therefore **memorises a probable noisy label**; a noise-tolerant method (minimum leaf size, pruning) leaves N2 as one misclassified example inside a larger leaf.

B. Re-evaluation of the original design (run with my implementation)

Experiment	Result
CE: E1–E5 then E6	S_5 and G_5 as before; E6 (identical to E1, label No): the only member of S covers E6, so S is removed and every G member is checked; **version space empty at step 6**.
CE: E1–E5 then E7	E7 positive (Medium, Regular, High, Weak, Passive, No): S generalised to ⟨?,?,?,?,Passive,?⟩ and G pruned to the same hypothesis; **S = G (single hypothesis, LMS = Passive)**.
CE: then E8	E8 positive (Low, Regular, High, Strong, **Active**, Yes) is not covered by the only hypothesis; S cannot be generalised within G → **version space empty**.

Decision tree	Before (14 records)	After N1–N3 (17 records)
Depth / nodes / leaves	3 / 13 / 8	4 / 18 / 11
Training accuracy	100 %	100 %
Change in structure	–	New branch "Very Low" under (Weak) with leaf Yes (N1); extra LMS split under (Strong, High, Regular) to isolate N2; Weak–Low leaf now has 4 Yes (N3)
Old tree on N1, N2, N3	–	N1 → Yes (back-off to node majority) ✓; N2 → No ✗ (missed at-risk student); N3 → Yes ✓

Handling N1: the original tree has no "Very Low" branch under (PriorCGPA = Weak, Attendance); the implemented policy returns the majority class of that node (Yes, 6 of 7), which is correct for N1. After retraining, N1 gets its own branch.

Requirements. Recall ≥ 90 %: CE fails (empty version space, no classifier). The fully grown tree reaches only 0.60 recall in leave-one-out and its training recall is perfect only because it memorises; the depth-1 tree scored 0.90 on the original 20 records but only 0.80 recall (8/10) on the 17-record training set because N2 is a Strong-CGPA at-risk student it cannot capture, so it is not safe either. **Explanations:** both approaches can explain (version-space hypothesis, or the rule path) but CE can do so only while the version space is non-empty. **Verdict:** neither original design remains suitable as it is.

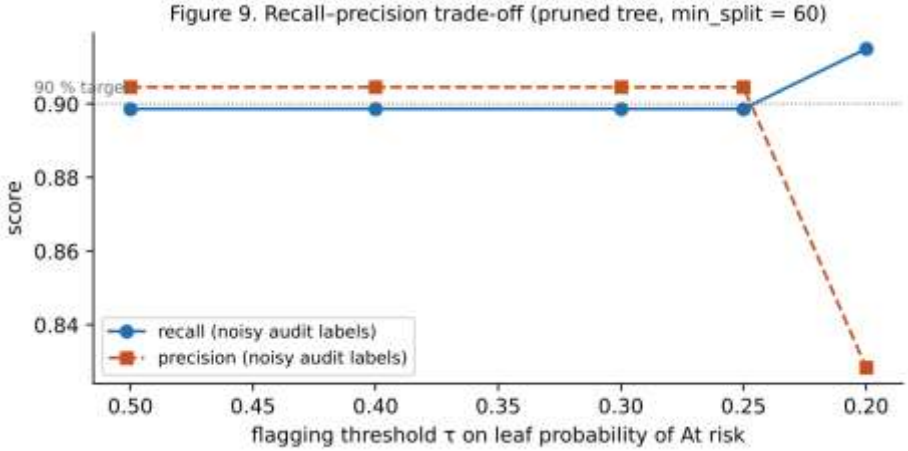
C. Adaptation or redesign: MODIFY (keep ID3, replace CE as classifier)

Retained	Modified	Replaced / role changed
Supervised paradigm; ID3 with entropy gain; tie-breaking by attribute order; rule extraction for explanations; seed-controlled pipeline	Stopping rule (min leaf 20 records ≈ 1 %, min gain 0.01); post-pruning on validation data; gain ratio as option; leaf probabilities with Laplace smoothing; class weight and threshold for recall	Candidate Elimination is no longer the classifier; it is kept as a **consistency auditor** to flag identical records with different labels (E6 type) for advisor adjudication. Hypothesis space changes from conjunctions to trees (disjunctions of conjunctions)

REVISED PIPELINE (run weekly on all labelled records)
1. AUDIT : group records by identical attribute values; if labels conflict -> send to advisor;
   if unresolved keep the majority label (ties -> At risk).
2. ENCODE : map new values by ordinal back-off (Very Low -> Low, kept as a flag) or to UNKNOWN.
3. SPLIT : stratified train / validation (and 5-fold CV for model selection).
4. BUILD(D): if pure, or |D| < 2*min_leaf, or best_gain < 0.01 -> Leaf(P(Yes) = (w*yes+1)/(w*yes+no+2))
   a* = argmax gain ratio over attributes (ties: fixed order); split; recurse.
5. PRUNE : reduced-error / cost-complexity pruning on the validation set.
6. THRESHOLD: choose tau = largest value with validation recall >= 0.93 (safety margin over 0.90).
7. PREDICT : follow the path; unseen value -> use distribution of the current node;
   flag if P(Yes) >= tau.
8. EXPLAIN : print rule path + counts ("12 of 14 similar students failed") + what would change it.

Unseen values and combinations: ordinal back-off for ordered attributes (Very Low behaves like Low plus a flag that the case is new) and the distribution of the current node otherwise; the number of back-off predictions is logged and triggers retraining when it exceeds a limit. **Recall requirement:** class weight w = 3 for the Yes class in leaf probabilities and a flag threshold τ below 0.5. Experiment on a larger synthetic set (1,400 training / 600 validation, 10 % label noise in both):

Figure 9. Recall-precision trade-off (pruned tree, min_split = 60)



Tree	Nodes	τ	Recall (noisy labels)	Precision (noisy labels)	Recall (true labels)	Precision (true labels)
Fully grown	183	0.5	0.8987	0.9046	1.0000	1.0000
Fully grown	183	0.2	0.9314	0.7830	1.0000	0.8352
Pruned (min_split = 60)	67	0.5	0.8987	0.9046	1.0000	1.0000
Pruned (min_split = 60)	67	0.2	0.9150	0.8284	1.0000	0.8994

Lowering τ from 0.5 to 0.2 raises recall measured against the noisy audit labels from 0.8987 to 0.9150 but lowers precision from 0.9046 to 0.8284 (about one flag in six is a false alarm). Against the true labels recall is already 1.0000, because 10 % of the labels are wrong and **no model can exceed about 90 % recall against noisy labels**; therefore the 90 % target should be verified on adjudicated labels. **Explanations:** every flagged student receives the matching rule, e.g. "Prior CGPA Weak and Attendance Low: 4 of 4 similar past students were at risk", with the leaf counts as evidence. **Expected effect:** about 3× fewer nodes than the full tree on noisy data (67 against 183), the same or better accuracy on clean data, recall adjustable by τ at a known precision cost.

D. Evidence-based defence

- **Learning theory and inductive bias:** the conjunctive bias is wrong for the new data (E7 then E8 show a disjunction, E8 cannot be covered); a tree can represent any Boolean function, so the bias problem disappears, and the remaining risk, variance, is controlled by pruning.
- **Version space under noise:** E6 empties the version space at step 6 and E8 at the next positive; Candidate Elimination has no way of recovering, which makes it unsuitable as a classifier for 5–10 % noise.
- **Heuristic search:** the greedy gain criterion built a consistent tree with only 14 gain computations, the Attendance-rooted alternative was not better (14 nodes, 33 % on T), and gain ratio changed nothing (0.9778 either way), so I keep information gain and only add the minimum-gain threshold; gain ratio becomes necessary only if multi-valued attributes are added.
- **Overfitting and underfitting:** full tree 100 % train / 50 % test and 133 nodes at 10 % noise; depth-1 tree 85.71 % / 83.33 % and 3 nodes; depth 0 underfits at 50 %. The controlled tree with min_split = 30 had 41 nodes and 100 % clean validation accuracy.
- **Recall requirement:** controlled tree LOO recall 0.90 against 0.60 for the full tree; threshold $\tau = 0.2$ gives 0.915 recall on noisy labels at precision 0.83.

Remaining limitation and monitoring. The model may drift (new online mode, new cohorts) and may be less accurate for some student groups; Dataset C is synthetic so thresholds are indicative only. After deployment I would: (i) compute recall weekly on newly labelled at-risk students in a rolling window of at least 200 and alert if its lower confidence bound falls below 0.90; (ii) track the share of back-off predictions (unseen values such as Very Low) and the distribution of each attribute (population stability) to trigger retraining; (iii) report recall and false-alarm rate per subgroup to check fairness; (iv) keep auditing 5 % of labels so the noise rate stays known.

APPENDIX A – Source code (ml.py)

```
"""ITA06 Assignment 1 - own implementations of Candidate Elimination and ID3. Seed fixed."""
import math, random, itertools
from collections import Counter
random.seed(42)
Q='?'; N='0'   # '?' any value, '0' = empty (∅)
# ---------- Concept learning ----------
# covers(h,x): True if hypothesis h matches instance x ('?' = any value, '0' = matches nothing)
def covers(h,x): return all(a==Q or (a!=N and a==v) for a,v in zip(h,x))
# more_general_eq(h1,h2): h1 >=g h2, i.e. every instance covered by h2 is covered by h1
def more_general_eq(h1,h2):
    if N in h2: return True
    return all(a==Q or a==b for a,b in zip(h1,h2))
# min_gen: minimal generalisation of S so that it covers positive example x
def min_gen(s,x):
    if s is None: return tuple(x)
    if all(a==N for a in s): return tuple(x)
    return tuple(a if a==v else Q for a,v in zip(s,x))
# min_spec: minimal specialisations of G that exclude negative example x
def min_spec(g,x,domains):
    out=[]
    for i,a in enumerate(g):
        if a==Q:
            for v in domains[i]:
                if v!=x[i]:
                    h=list(g); h[i]=v; out.append(tuple(h))
    return out
# Candidate Elimination: S and G are sets; prints S and G after each example; stops if version space is empty
def candidate_elimination(data,domains,verbose=True):
    n=len(domains)
    S={tuple([N]*n)}; G={tuple([Q]*n)}
    trace=[(0,None,None,set(S),set(G),'init')]
    for k,(x,y) in enumerate(data,1):
        if y:
            G={g for g in G if covers(g,x)}
            S2=set()
            for s in S:
                if covers(s,x): S2.add(s)
                else:
                    h=min_gen(s,x)
                    if any(more_general_eq(g,h) for g in G): S2.add(h)
            S={s for s in S2 if not any(s!=t and more_general_eq(s,t) for t in S2)}
        else:
            S={s for s in S if not covers(s,x)}
            G2=set()
            for g in G:
                if not covers(g,x): G2.add(g)
                else:
                    for h in min_spec(g,x,domains):
                        if any(more_general_eq(h,s) for s in S): G2.add(h)
            G={g for g in G2 if not any(g!=t and more_general_eq(t,g) for t in G2)}
        trace.append((k,x,y,set(S),set(G),''))
        if verbose: print(k,'+' if y else '-',sorted(S),sorted(G))
        if not S or not G:
            if verbose: print('VERSION SPACE EMPTY at example',k)
            return S,G,trace,k
    return S,G,trace,None
def version_space(S,G,domains):
    hs=itertools.product(*[list(d)+[Q] for d in domains])
    return [h for h in hs if any(more_general_eq(h,s) for s in S) and any(more_general_eq(g,h) for g in G) and N not in h]
# ---------- Decision tree ----------
# entropy of a list of class labels (base 2)
def entropy(labels):
    c=Counter(labels); n=len(labels)
    return abs(-sum(v/n*math.log2(v/n) for v in c.values() if v)) if n else 0.0
# information gain of attribute index a
def gain(rows,labels,a):
    n=len(rows); e=entropy(labels); w=0
    for v in set(r[a] for r in rows):
        sub=[l for r,l in zip(rows,labels) if r[a]==v]
        w+=len(sub)/n*entropy(sub)
    return e-w
def split_info(rows,a):
    n=len(rows); c=Counter(r[a] for r in rows)
    return -sum(v/n*math.log2(v/n) for v in c.values())
class Node:
    def __init__(s,**k): s.__dict__.update(k)
def majority(labels,tie='Yes'):
```

```
    c=Counter(labels)
    if c['Yes']==c['No']: return tie   # tie -> Yes (favour recall)
    return c.most_common(1)[0][0]
# ID3: stops when pure / no attributes / max_depth / min_split; ties in gain -> first attribute in tie_order
def id3(rows,labels,attrs,names,depth=0,max_depth=None,min_split=2,min_gain=0.0,tie_order=None,measure='gain',log=None):
    if len(set(labels))==1: return Node(leaf=labels[0],n=Counter(labels))
    if not attrs or (max_depth is not None and depth>=max_depth) or len(rows)<min_split:
        return Node(leaf=majority(labels),n=Counter(labels))
    sc={}
    for a in attrs:
        g=gain(rows,labels,a)
        if measure=='ratio':
            si=split_info(rows,a); g=g/si if si>0 else 0
        sc[a]=g
    best=max(attrs,key=lambda a:(round(sc[a],10),-(tie_order or attrs).index(a)))
    if log is not None: log.append((depth,len(rows),dict(Counter(labels)),{names[a]:sc[a] for a in attrs},names[best]))
    if sc[best]<=min_gain and min_gain>0 or sc[best]<=0:
        return Node(leaf=majority(labels),n=Counter(labels))
    node=Node(attr=best,children={},default=majority(labels),n=Counter(labels))
    for v in sorted(set(r[best] for r in rows)):
        idx=[i for i,r in enumerate(rows) if r[best]==v]
        node.children[v]=id3([rows[i] for i in idx],[labels[i] for i in idx],[a for a in attrs if
a!=best],names,depth+1,max_depth,min_split,min_gain,tie_order,measure,log)
    return node
# predict: unseen attribute value -> majority class of the current node (policy for TC7)
def predict(t,x):
    while not hasattr(t,'leaf'):
        if x[t.attr] in t.children: t=t.children[x[t.attr]]
        else: return t.default   # unseen value policy: node majority
    return t.leaf
def stats(t):
    if hasattr(t,'leaf'): return 0,1,1   # depth,nodes,leaves
    r=[stats(c) for c in t.children.values()]
    return 1+max(x[0] for x in r),1+sum(x[1] for x in r),sum(x[2] for x in r)
def rules(t,names,pre=()):
    if hasattr(t,'leaf'):
        return ['IF '+(' AND '.join(pre) if pre else 'TRUE')+' THEN AtRisk = '+t.leaf+'   [%s]'%dict(t.n)]
    out=[]
    for v,c in t.children.items(): out+=rules(c,names,pre+('%s = %s'%(names[t.attr],v),))
    return out
def show(t,names,ind=0):
    if hasattr(t,'leaf'): print(' '*ind+'-> '+t.leaf,dict(t.n)); return
    for v,c in t.children.items():
        print(' '*ind+'%s = %s'%(names[t.attr],v)); show(c,names,ind+4)
def acc(t,rows,labels): return sum(predict(t,r)==l for r,l in zip(rows,labels))/len(rows)
# ---------- Data ----------
DOM_A=[('Low','Medium','High'),('Irregular','Regular'),('Low','High'),('Weak','Strong'),('Passive','Active'),('No','Yes')]
A=[(('Low','Irregular','Low','Weak','Passive','No'),1),(('Low','Irregular','Low','Strong','Passive','No'),1),
  (('High','Regular','High','Strong','Active','Yes'),0),(('Low','Irregular','High','Weak','Passive','Yes'),1),
  (('Medium','Irregular','High','Strong','Active','No'),0)]
NM=['Attendance','Assignments','LMS','PriorCGPA']
B=[('Low','Irregular','Passive','Weak','Yes'),('Low','Irregular','Passive','Strong','Yes'),('Low','Regular','Active','Weak','Yes'),
('Low','Regular','Active','Strong','No'),('Medium','Irregular','Passive','Weak','Yes'),('Medium','Irregular','Active','Strong','No'),
('Medium','Regular','Passive','Weak','Yes'),('Medium','Regular','Active','Strong','No'),('High','Irregular','Passive','Weak','Yes'),
('High','Irregular','Active','Strong','No'),('High','Regular','Passive','Strong','No'),('High','Regular','Active','Weak','No'),
('Medium','Irregular','Passive','Strong','No'),('Low','Irregular','Active','Weak','Yes')]
T=[('Low','Irregular','Active','Strong','No'),('Medium','Regular','Active','Weak','No'),('High','Regular','Passive','Weak','Yes'),
('Medium','Irregular','Active','Weak','Yes'),('Low','Regular','Passive','Strong','No'),('High','Irregular','Active','Weak','Yes')]
XB=[r[:4] for r in B]; YB=[r[4] for r in B]; XT=[r[:4] for r in T]; YT=[r[4] for r in T]
```

APPENDIX B – Test script (tests.py)

```
import sys; sys.path.insert(0,'/home/claude/work')
from ml import *
res=[]
def chk(name,ok,obs): res.append((name,ok,obs)); print(name,'PASS' if ok else 'FAIL',obs)
S,G,tr,col=candidate_elimination(A,DOM_A,verbose=False)
exp_S5={('Low','Irregular','?','?','Passive','?')}; exp_G5={('Low','?','?','?','?','?'),('?','?','?','?','?','Passive','?')}
chk('TC1',S==exp_S5 and G==exp_G5 and col is None,'S5=%s G5=%s'%(sorted(S),sorted(G)))
chk('TC2a',entropy(['Yes']*5)==0,'H(pure)=%.4f'%entropy(['Yes']*5)); chk('TC2b',abs(entropy(['Yes']*7+['No']*7)-1)<1e-
12,'H(7/7)=%.4f'%entropy(['Yes']*7+['No']*7))
g=[round(gain(XB,YB,a),4) for a in range(4)]; chk('TC3',g==[0.1636,0.0611,0.1369,0.4083],str(dict(zip(NM,g))))
t=id3(XB,YB,[0,1,2,3],NM); chk('TC4',acc(t,XB,YB)==1.0 and stats(t)==(3,13,8) and len(rules(t,NM))==8,'train acc=%.4f stats(depth,nodes,leaves)=%s
rules=%d'%(acc(t,XB,YB),stats(t),len(rules(t,NM))))
p=[predict(t,x) for x in XT]; chk('TC5',acc(t,XT,YT)==0.5,'preds=%s test acc=%.4f'%(p,acc(t,XT,YT)))
_,_,_,c6=candidate_elimination(A+[(('Low','Irregular','Low','Weak','Passive','No'),0)],DOM_A,verbose=False); chk('TC6',c6==6,'version space empty reported at
example %s (no crash)'%c6)
```

```
q=('Very Low','Irregular','Passive','Weak'); chk('TC7',predict(t,q)=='Yes','policy: missing branch -> majority class of current node; prediction=%s'%predict(t,q))
lg=[]; id3(XB,YB,[0,1,2,3],NM,log=lg); tie=[l for l in lg if l[0]==2];
t_a=id3(XB,YB,[0,1,2,3],NM); t_b=id3(XB,YB,[0,1,2,3],NM,tie_order=[0,1,2,3]); same=rules(t_a,NM)==rules(t_b,NM)
chk('TC8',same and all(l[4]=='Assignments' for l in tie),'gain(Assignments)=gain(LMS)=1.0000 at depth-2 nodes; chosen=%s; repeat runs
identical=%s'%(sorted(set(l[4] for l in tie)),same))
print(all(r[1] for r in res))
```